

The Hong Kong Polytechnic University

COMP1012 Programming Fundamentals and Applications

Project Alpha: Adaptive EEG QC Python ABI Integration

Level of Difficulty: Advanced

Academic Year 2026–2027

Contents

1	Project Overview	3
1.1	Introduction	3
1.2	Learning Objectives	3
1.3	Alignment with COMP1012	4
1.4	Difficulty Benchmark	4
1.5	System Architecture	4
1.6	Required Implementation	5
1.7	What Is Provided	5
2	Technical Requirements	6
2.1	Data Models	6
2.1.1	Window Payload	6
2.1.2	Model Input Vector	6
2.1.3	Plugin Output	6
2.2	Component 1: ABI Input and Model Artifact Loading	6
2.2.1	Overview	6
2.2.2	Required Behavior	6
2.2.3	Interface Specification	7
2.2.4	Evaluation Criteria	7
2.3	Component 2: ML QC Decision Hook	7
2.3.1	Overview	7
2.3.2	Required Behavior	7
2.3.3	Example Skeleton	7
2.3.4	Evaluation Criteria	8
2.4	Component 3: ML Feature Extraction Hook	8
2.4.1	Overview	8
2.4.2	Required Behavior	8
2.4.3	Example Skeleton	8
2.4.4	Evaluation Criteria	9
2.5	Component 4: Output, Testing, and Integration	9
2.5.1	Overview	9
2.5.2	Required Behavior	9
2.5.3	Evaluation Criteria	9
3	Python Plugin Architecture	10
3.1	Plugin Package Structure	10
3.2	Model Manifest	10
3.3	Testing Workflow	11
3.4	ML Safety Constraints	11

4	Provided Runtime and ABI Files	12
4.1	Bundled ABI Files	12
4.2	Running the Checks	12
4.3	Running a Plugin	12
4.4	What Is Abstracted	12
5	Deliverables	14
5.1	Source Code	14
5.2	Written Report	14
5.2.1	Required Sections	14
6	Evaluation Summary	16
6.1	Grading Rubric	16
6.1.1	Python ML Plugin Functionality (50%)	16
6.1.2	Written Report (30%)	16
6.1.3	Optional QA Checkpoints (5% each)	17
6.1.4	Optional Advanced Modules (up to 20%)	17
7	Resources	19
7.1	Python Programming	19
7.2	Machine Learning and Numeric Processing	19
7.3	Project ABI Workflow	19

Chapter 1

Project Overview

1.1 Introduction

In this project, students will build a **Python machine-learning quality-control plugin** for an Adaptive EEG QC pipeline. The provided host supplies a stable Bash/CSV Python ABI: it reads EEG-like CSV recordings, separates EEG and optional EOG sidecar channels, creates fixed windows, assigns canonical IDs, and exports window metadata and sample CSV tables. Students implement Python hooks that load or run an ML model and return validated QC decisions and numeric features.

Project Alpha and Project Beta are alternative project options. Students should complete only one of them.

Project Alpha uses **ML-based inference**. Students may use a trained classifier, anomaly detector, embedding model, shallow neural network, tree model, or other learned method. The ML model must remain outside the provided ABI runtime and must be called only through the Python ABI boundary. Training labels, fitted scalars, model parameters, and evaluation methodology must be documented clearly to prevent data leakage.

The focus is on the **COMP1012 programming fundamentals** needed to solve an applied data-processing problem:

- Reading structured CSV data and preserving window identity
- Using Python lists, dictionaries, strings, and numeric values correctly
- Designing functions for feature building, model loading, and inference
- Applying selection, repetition, file operation, and modular program design
- Producing host-checkable output rows with model provenance

1.2 Learning Objectives

By completing this project, students will demonstrate understanding of:

- **Python program structure:** Implement readable modules, functions, and tests
- **Data representation:** Use lists and dictionaries to represent windows, features, and metadata
- **File and CSV operation:** Read provided inputs and write validated outputs
- **Flow control:** Process many windows and map scores to decisions logically
- **Model provenance:** Record model version, threshold, scaler, and feature list
- **Problem solving:** Separate training assumptions from inference-time Python code

1.3 Alignment with COMP1012

This assignment is designed for **COMP1012 Programming Fundamentals and Applications**. The subject description states that COMP1012 introduces fundamental programming elements and the use of programming to solve practical problems in different domains. This project therefore treats EEG quality control as an applied Python programming problem rather than an operating-systems project.

The required work maps directly to the COMP1012 indicative syllabus.

1.4 Difficulty Benchmark

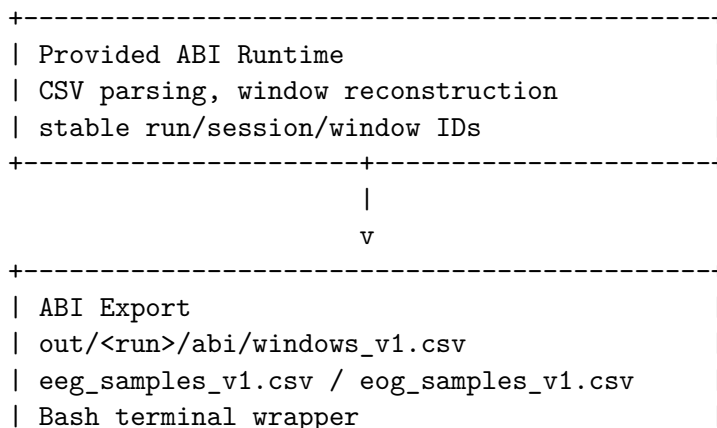
This project is intended to match the workload and difficulty of a Year 1 semester project built around a modular applied Python system. The supplied passline example is deliberately the simplest acceptable code implementation. Passing the passline benchmark means the code can load through the ABI, return valid output, and perform an initial identification of EEG data quality on the supplied examples. Passline corresponds to the 60% baseline for the code component. Higher marks should come from clearer design, better features, stronger tests, better explanations, justified extensions, and BCI IV 2a benchmark analysis, not from changing the ABI.

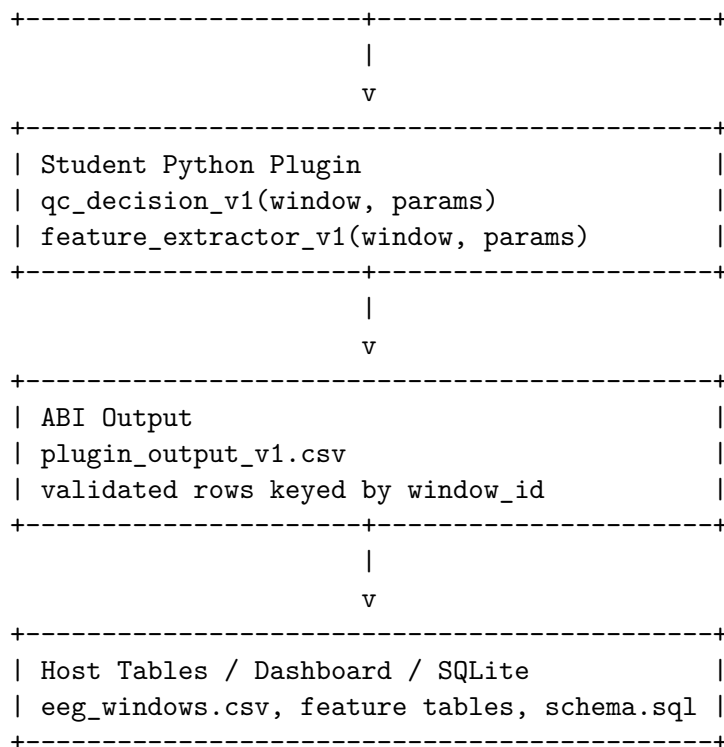
The BCI IV 2a benchmark grading points are sensitivity, specificity, precision, F1, and balanced accuracy.

The intended module-level difficulty is comparable to a project with separate question-bank, response-tracking, adaptive-learning, feedback, visualization, and learner-modeling modules. In this EEG QC project, the corresponding modules are:

General Y1 Module Type	EEG QC Project Equivalent
Question/data bank	ABI windows and EEG/EOG sample CSV inputs
Response/performance tracking	Per-window QC outputs and feature rows
Adaptive decision logic	Model thresholds, confidence mapping, and parameterized inference
Feedback/recommendation	Reason codes and report discussion of false keep/false reject risks
Visualization	Progress, feature, or error-analysis plots in the report
Learner/system modeling	Window-quality profiling, benchmark cases, and model provenance

1.5 System Architecture





1.6 Required Implementation

Component	Description
ABI Input Reader	Stream and validate <code>windows_v1.csv</code> records
Feature Builder	Convert raw window samples into model input features
ML Inference Hook	Load a trained model or detector and produce QC status
Output Writer	Write valid <code>plugin_output_v1.csv</code> rows keyed by <code>window_id</code>
Provenance and Safety	Report model name, version, parameters, and failure modes
Written Report	Academic report and reproducible run evidence
Optional QA Checkpoints	Up to three staged reviews of current implementation progress
Optional Advanced Modules	ML-system extensions beyond the core requirements

1.7 What Is Provided

The package provides the ABI runner, sample ABI CSV files, a passing-line plugin, tests, and template documentation. Full paths and commands are listed once in Chapter 4 to avoid duplicating setup instructions.

Chapter 2

Technical Requirements

2.1 Data Models

2.1.1 Window Payload

The host writes one window metadata row in `windows_v1.csv` and sample rows in `eeg_samples_v1.csv` plus optional `eog_samples_v1.csv`. The ML plugin must treat these reconstructed windows as inference input only. Ground-truth labels must not be required in any ABI CSV file.

2.1.2 Model Input Vector

Students must define a deterministic feature vector before inference. Examples include per-channel amplitude features, variance, band-power estimates, correlation summaries, or learned embeddings. The feature order must be stable and recorded in a manifest.

2.1.3 Plugin Output

The plugin writes `plugin_output_v1.csv` rows using `qc_decision_v1` and optionally `feature_extractor_v1`. All rows must preserve the original `window_id`. QC output should include a status and may include confidence-like scores through `quality_score`.

2.2 Component 1: ABI Input and Model Artifact Loading

2.2.1 Overview

The plugin must initialize the model and then process window payloads. Initialization should be explicit and repeatable. Missing model files, incompatible feature lists, or unsupported ABI versions should fail before partial results are written.

2.2.2 Required Behavior

1. Use the provided ABI runner to load CSV tables and validate `abi_version`
2. Load model artifacts from declared paths, not hidden global state
3. Load any scaler, threshold, label map, or feature manifest needed for inference
4. Reject duplicate `window_id` values

5. Preserve host identity fields for every output row

2.2.3 Interface Specification

A typical ML plugin should expose module-level initialization through lazy loading:

```
1 _MODEL = None
2
3
4 def load_model(params):
5     global _MODEL
6     if _MODEL is None:
7         path = params["model_path"]
8         _MODEL = load_artifact(path) # student-defined loader
9     return _MODEL
```

2.2.4 Evaluation Criteria

Criterion	Description
Artifact loading	Model, scaler, and metadata load predictably
ABI validation	Unsupported records fail clearly
Identity preservation	Output rows retain valid host window_id values
No leakage	Inference does not require ground-truth labels

2.3 Component 2: ML QC Decision Hook

2.3.1 Overview

The QC hook runs model inference and maps model output to ABI statuses. The mapping from probability, anomaly score, or classifier output to KEEP, WARN, REJECT, or ABSTAIN must be documented and configurable.

2.3.2 Required Behavior

1. Implement `qc_decision_v1(window, params)`
2. Build the exact feature vector expected by the model
3. Apply the same preprocessing used during training
4. Map model output to an ABI status using declared thresholds
5. Return reason codes that identify model path, threshold family, or dominant feature group

2.3.3 Example Skeleton

```
1 def qc_decision_v1(window, params):
2     model = load_model(params)
3     x = build_feature_vector(window, params)
4     score = float(model.predict_proba([x])[0][1])
5
6     reject_t = float(params.get("reject_probability", 0.90))
7     warn_t = float(params.get("warn_probability", 0.60))
8
```



```

9     if score >= reject_t:
10         status = "REJECT"
11     elif score >= warn_t:
12         status = "WARN"
13     else:
14         status = "KEEP"
15
16     quality = max(0.0, min(100.0, 100.0 * (1.0 - score)))
17     return {"status": status, "quality_score": quality,
18           "reason_codes": ["ML_SCORE"]}

```

2.3.4 Evaluation Criteria

Criterion	Description
Inference correctness	Model receives the expected feature vector shape
Threshold clarity	Status mapping is documented and reproducible
Provenance	Model version and parameters are reported in the report
Robustness	Missing artifacts and invalid model outputs fail cleanly

2.4 Component 3: ML Feature Extraction Hook

2.4.1 Overview

The feature hook should expose interpretable features used by the model, model-ready transformed values, or additional diagnostic features. This is important because the host can store and compare numeric features even when model decisions change.

2.4.2 Required Behavior

1. Implement `feature_extractor_v1(window, params)`
2. Return finite numeric features with stable names
3. Keep feature order and naming consistent with the model manifest
4. Include at least a small set of interpretable features even if the model uses transformed inputs
5. Avoid returning raw labels or training-only values

2.4.3 Example Skeleton

```

1 def feature_extractor_v1(window, params):
2     x = build_feature_vector(window, params)
3     names = params.get("feature_names", [])
4     if names and len(names) == len(x):
5         return {name: float(value) for name, value in zip(names, x)}
6     return {f"ml_feature_{i}": float(value) for i, value in enumerate(x)}

```

2.4.4 Evaluation Criteria

Criterion	Description
Feature manifest	Feature names match the model documentation
Numeric safety	No NaN, infinity, or non-numeric values
Interpretability	At least some returned features explain model decisions
Efficiency	Feature extraction avoids excessive memory copies

2.5 Component 4: Output, Testing, and Integration

2.5.1 Overview

The output component writes validated CSV rows. The ML version has additional testing requirements because model artifacts and dependency versions can silently change behavior.

2.5.2 Required Behavior

1. Produce valid ABI rows for every successfully processed window
2. Include a separate manifest in the submission describing model file, feature list, thresholds, and training split
3. Provide tests for clean, noisy, flatline, malformed, and out-of-distribution windows
4. Demonstrate repeatable inference from a fresh process
5. Keep all model artifacts outside canonical ABI input and output files

2.5.3 Evaluation Criteria

Criterion	Description
Valid output	All rows conform to <code>plugin_output_v1.contract.csv</code>
Model reproducibility	Fresh-process runs produce stable decisions
Artifact documentation	Model, scaler, features, and thresholds are documented
Integration	Runs against generated <code>windows_v1.csv</code> without host changes

Chapter 3

Python Plugin Architecture

3.1 Plugin Package Structure

Students should submit a project folder containing the plugin module, tests, model artifacts or artifact-download instructions, preprocessing metadata, and a short manifest. The plugin module must be loadable by the provided ABI runner without modifying the bundled runtime.

```
student_ml_plugin/  
  plugin.py  
  params.json  
  model_manifest.json  
  feature_manifest.json  
  README.md  
tests/  
  test_plugin.py
```

3.2 Model Manifest

Every ML submission must include a manifest describing:

- Model family and library version
- Model artifact path or reproducible download/build instructions
- Feature names and feature order
- Training, validation, and test split policy
- Decision thresholds for KEEP, WARN, and REJECT
- Known failure modes and expected input shape

```
1 {  
2   "model_name": "student_eeg_qc_detector",  
3   "model_version": "0.1.0",  
4   "feature_manifest": "feature_manifest.json",  
5   "warn_probability": 0.60,  
6   "reject_probability": 0.90  
7 }
```

3.3 Testing Workflow

Students must test the plugin at four levels:

1. **Unit tests:** Direct calls to feature builder and hooks
2. **Artifact tests:** Verify model and preprocessing files can be loaded
3. **ABI tests:** Run the helper on a generated `windows_v1.csv`
4. **Repeatability tests:** Run inference from a fresh process twice and compare output rows

3.4 ML Safety Constraints

The following are required in Project Alpha:

- Ground-truth labels must not be read during inference
- Training data and benchmark/test data must be separated in the report
- Model output must be mapped to the ABI status enum explicitly
- Missing model artifacts must fail with a clear error before partial output is written

Chapter 4

Provided Runtime and ABI Files

4.1 Bundled ABI Files

The following files define the terminal-facing Python interface in this passline package:

- `abi_runtime/eeg_qc_abi.py`: Python-only ABI runner and output validation
- `ABI_TEMPLATE.md`: hook names, signatures, return contract, and command examples
- `sample_abi/windows_v1.csv`: sample window metadata
- `sample_abi/eeg_samples_v1.csv`: sample EEG table
- `sample_abi/eog_samples_v1.csv`: sample EOG table
- `sample_output/plugin_output_v1.csv`: generated plugin output example

4.2 Running the Checks

```
1 python3 scripts/benchmark_complex.py
```

This command runs unit tests, direct benchmark cases, and ABI integration checks using the bundled sample files.

4.3 Running a Plugin

```
1 python3 abi_runtime/eeg_qc_abi.py \  
2 --windows sample_abi/windows_v1.csv \  
3 --eeg-samples sample_abi/eeg_samples_v1.csv \  
4 --eog-samples sample_abi/eog_samples_v1.csv \  
5 --plugin student_ml_plugin/plugin.py \  
6 --out sample_output/plugin_output_v1.csv
```

Students may replace `student_ml_plugin/plugin.py` with their own implemented plugin path as long as the ABI hook names and output contract are preserved.

4.4 What Is Abstracted

Students do **not** need to implement the following:

- ABI CSV discovery, parsing, or window reconstruction
- Generation of `run_id`, `session_id`, or `window_id`
- Dashboard scaffolding or SQLite loading
- Direct modification of canonical host CSV files

Students focus on the Python algorithm and on producing validated ABI rows.

Chapter 5

Deliverables

5.1 Source Code

Submit a Python plugin folder that can run with the supplied ABI runner. Do not submit changes to the bundled runtime as the main solution.

- `plugin.py` implementing `qc_decision_v1` and `feature_extractor_v1`
- `model_manifest.json` describing model family, version, thresholds, and expected feature shape
- `feature_manifest.json` listing feature names and feature order
- Model artifact files or reproducible build/download instructions
- Unit tests for feature building, model loading, decision mapping, and invalid inputs
- One ABI integration test using `sample_abi/*.csv` or instructor-provided ABI files
- `README.md` explaining how to run the plugin from the project root
- A small sample `plugin_output_v1.csv` generated by the submitted plugin

The submitted `README.md` should include the exact command used to run the plugin and identify the generated `plugin_output_v1.csv`.

5.2 Written Report

The report must follow the academic structure and length requirements below while explaining the programming design, model input construction, and inference-time behavior at a level suitable for COMP1012.

5.2.1 Required Sections

1. **Abstract** (800–1100 characters):
 - Concise summary of the project
 - The feature design, model approach, and inference policy
 - Key results from the passline tests and any BCI IV 2a evaluation
2. **Introduction** (600–1000 words):
 - Context on EEG quality control
 - Project objectives
 - Overview of the Python plugin and supplied ABI workflow

3. **Related Works** (600–1400 words):

- Survey of relevant EEG quality-control and artifact-detection approaches
- Brief overview of relevant machine-learning methods
- Comparison with the chosen feature and model approach

4. **Implementation** (1400–2000 words):

- Separate subsections for ABI input and artifact loading, feature construction, ML decision logic, and output validation
- Model family, training or frozen-model rationale, thresholds, provenance, and status mapping
- Design choices and their justification
- Code snippets for critical sections

5. **Benchmark** (1000–1400 words):

- Test methodology using unit tests, ABI integration, and repeatability checks
- Results from the supplied examples and, where completed, the BCI IV 2a benchmark
- Sensitivity, specificity, precision, F1, and balanced accuracy where applicable
- Representative failure cases, tables, and graphs

6. **Discussion** (minimum 1000 words):

- Analysis of results
- Trade-offs in feature, model, and threshold choices
- Data leakage risks and limitations
- Comparison with alternative approaches
- Lessons learned about Python design and reproducible evaluation

7. **Conclusion and Future Work** (minimum 600 words):

- Summary of achievements
- How project objectives were met
- Potential improvements
- Future extensions

8. **References**:

- Minimum 20 references in APA style
- Include Python and ABI documentation, EEG quality-control literature, and relevant machine-learning sources

Chapter 6

Evaluation Summary

The compulsory assessment accounts for **80%**: 50% for Plugin Functionality and 30% for the Written Report. The remaining **20% is optional**. It may be earned through QA checkpoints, Advanced Modules, or a combination of the two, with the combined optional contribution capped at 20%.

Component	Weight	Key Criteria
Python ML Plugin Functionality	50%	Correct hooks, valid ABI output, feature vector, inference, provenance, tests
Written Report	30%	Academic quality, complete sections, design and evaluation
Optional QA and Advanced Modules	Up to 20%	5% per completed QA checkpoint plus Advanced Module marks, including justified BCI IV 2a analysis; combined optional marks are capped at 20%
Maximum compulsory mark	80%	Plugin Functionality and Written Report
Maximum project mark	100%	Compulsory and optional marks combined

6.1 Grading Rubric

6.1.1 Python ML Plugin Functionality (50%)

- ABI input and artifact loading (8%): validates IDs and loads declared model/scaler/manifest files predictably
- Feature builder (9%): stable feature vector order, numeric safety, clear handling of invalid windows
- ML decision hook (13%): correct status enum, reproducible threshold mapping, useful reason codes
- Output contract and provenance (8%): valid `plugin_output_v1.csv` rows with documented model metadata
- Testing (6%): unit, artifact, ABI, edge-case, and repeatability tests
- Code quality (6%): readable Python, modular functions, appropriate comments, no hidden training-time behavior

6.1.2 Written Report (30%)

- Title (unique and descriptive): 1%
- Abstract quality: 3%

- Introduction completeness: 3%
- Related works depth: 3%
- Implementation detail: 6%
- Benchmark thoroughness: 5%
- Discussion depth: 4%
- Conclusion and future work: 3%
- References quality: 2%

6.1.3 Optional QA Checkpoints (5% each)

Three optional QA checkpoints are available, each worth **5%**. You may complete zero, one, two, or all three, subject to the published arrangements. For each checkpoint, complete the supplied Code Update Report and relevant detail sheets, bring the current project files, demonstrate the milestone to the reviewer, and answer short questions about the implementation. No video submission is required.

Checkpoint	Optional mark	Evidence for this stage
Checkpoint 1	5%	The environment is ready, the plugin imports, and the required ABI hooks and model metadata are present.
Checkpoint 2	5%	Feature construction and ML decision logic work on sample and edge-case windows, with design choices explained.
Checkpoint 3	5%	The exact reviewed files pass the supplied unit, ABI, and repeatability checks and produce valid output rows.

The Code Update Report is considered first. A complete and internally consistent report supported by the reviewed code may receive the full checkpoint mark even when improvements are identified during QA. QA verifies code ownership and provides revision advice; receiving that advice does not by itself reduce the mark.

QA marks and Advanced Module marks are added together and capped at **20%**. Three completed QA checkpoints contribute 15%, leaving at most 5% available from Advanced Modules toward the final project grade.

6.1.4 Optional Advanced Modules (up to 20%)

Advanced marks may be earned through clearly documented extensions and BCI IV 2a benchmark analysis. The benchmark grading points are sensitivity, specificity, precision, F1, and balanced accuracy. You may combine QA and Advanced Module marks in any proportion, but together they cannot contribute more than 20%. These are suggestions, not compulsory requirements:

- Add at least five useful signal-quality features rather than relying only on minimal range/flatline examples.
- Emit richer reason codes, such as AMP_RANGE, FLATLINE, LOW_VARIANCE, and EOG_ARTIFACT, when supported by evidence.
- Handle malformed, empty, partial, or unusual windows clearly and reproducibly.
- Submit additional student-designed test cases beyond the bundled examples.
- Include failure analysis explaining where the approach works and where it can fail.

- Run the BCI IV 2a benchmark and explain the five grading metrics and representative failure cases.
- For ML submissions, document the training/evaluation split or explain the frozen model rationale and calibration choices.

Chapter 7

Resources

7.1 Python Programming

- Python csv module documentation: <https://docs.python.org/3/library/csv.html>
- Python pathlib documentation: <https://docs.python.org/3/library/pathlib.html>
- Python tutorial: <https://docs.python.org/3/tutorial/>

7.2 Machine Learning and Numeric Processing

- NumPy documentation: <https://numpy.org/doc/stable/>
- Scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html
- MNE-Python tutorials: https://mne.tools/stable/auto_tutorials/index.html

7.3 Project ABI Workflow

- Project file: `ABI_TEMPLATE.md`
- Project file: `abi_runtime/eeg_qc_abi.py`
- Project folder: `sample_abi/`